DOCUMENTACIÓN TÉCNICA Y DIDÁCTICA

Plataforma EMSE

Estado completo del sistema hasta julio de 2026

Arquitectura · PostgreSQL · Prisma · Migraciones · NestJS · Next.js · Seguridad · Panel administrativo · Herramientas

Documento generado desde el código fuente actual

Índice

1. Resumen del sistema
2. Arquitectura y recorrido de una solicitud
3. Herramientas y tecnologías
4. Estructura del repositorio
5. Historia completa de migraciones
6. Esquema Prisma: enumeraciones y modelos
7. Backend NestJS: clases y responsabilidades
8. Autenticación, OTP y seguridad
9. Frontend Next.js: páginas, componentes y servicios
10. Panel administrativo actual
11. Seed y datos actuales
12. Pruebas y comandos
13. Qué está terminado y qué queda pendiente
14. Glosario

1. Resumen del sistema

La Plataforma EMSE es una aplicación web institucional dividida en un frontend Next.js, una API REST NestJS y una base PostgreSQL administrada mediante Prisma. Actualmente el trabajo funcional está concentrado en control de acceso y panel administrativo. La estructura de notas existe en la base como preparación futura, pero el flujo académico de notas todavía no debe considerarse implementado.

> **Alcance actual:** autenticación en dos pasos, sesiones seguras, usuarios, cadetes, cursos, roles, permisos, secciones, jerarquías, asignaciones académicas estructurales y auditoría. No incluye todavía registro, aprobación, ponderación o publicación de notas.

Capa	Responsabilidad	Tecnología
Presentación	Páginas, formularios, tablas, navegación, estados visuales y consumo de API.	Next.js 16, React 19, TypeScript, Tailwind CSS
Aplicación	Reglas de negocio, validación, autorización, transacciones y endpoints.	NestJS 11, class-validator, JWT
Datos	Persistencia relacional, integridad, índices, historiales y transacciones.	PostgreSQL, Prisma 7, pg
Correo	Envío del OTP institucional y reporte SMTP.	Nodemailer, SMTP

2. Arquitectura y recorrido de una solicitud

```
Usuario en navegador
    ↓ interacción
Página/Componente React (frontend/src)
    ↓ servicio Axios + Bearer token
Controller NestJS
    ↓ Guard JWT → Guard de permisos → DTO validado
Service NestJS (reglas de negocio)
    ↓ PrismaService / transacción
PostgreSQL
    ↑ respuesta { data, meta } o entidad
Frontend actualiza únicamente la tabla/pantalla afectada
```

Los **controllers** definen URLs y reciben datos. Los **DTO** verifican que esos datos tengan formato permitido. Los **guards** impiden el acceso sin sesión o permiso. Los **services** contienen las reglas de negocio. Prisma convierte operaciones TypeScript en SQL seguro.

3. Herramientas y tecnologías

Herramienta	Para qué se usa
Node.js / npm	Ejecuta backend y frontend, instala dependencias y lanza scripts.
TypeScript	Añade tipos a JavaScript y detecta incompatibilidades durante compilación.
NestJS	Organiza la API en módulos, controllers, services, DTO, guards y filtros.
Next.js / React	Construye la interfaz, rutas y componentes interactivos del navegador.
Prisma ORM	Describe el modelo relacional, genera el cliente y administra migraciones.
PostgreSQL	Base de datos relacional definitiva.
bcrypt	Hash irreversible de contraseñas. Nunca debe almacenarse la contraseña clara.
JWT	Access token y refresh token para mantener sesiones autenticadas.
HMAC-SHA-256	Protege el OTP almacenando un hash verificable en vez del código claro.
Nodemailer	Conecta con SMTP y envía el correo OTP real.
Axios	Cliente HTTP del frontend; agrega el token a cada solicitud.
Jest	Pruebas unitarias del backend.
Swagger	Documentación interactiva de endpoints en `/api/documentacion`.
Git	Control de versiones; permite revisar cambios y recuperar versiones anteriores.

4. Estructura del repositorio

```
plataforma-EMSE/
├─ backend/
│  ├─ prisma/schema.prisma      modelo actual
│  ├─ prisma/migrations/        historia SQL incremental
│  ├─ prisma/seed/              catálogos y 363 cadetes
│  └─ src/                      API NestJS
├─ frontend/
│  ├─ src/app/                  rutas y páginas Next.js
│  ├─ src/componentes/          UI reutilizable
│  ├─ src/servicios/            llamadas Axios
│  └─ src/tipos/                contratos TypeScript
└─ documentación y respaldos
```

5. Historia completa de migraciones

Una migración es una modificación versionada de la estructura de la base. Se ejecutan en orden. No son datos de prueba y no deben borrarse. `prisma migrate reset` destruiría y recrearía la base, por eso no se usa con los 363 cadetes.

1) 20260518194003_control_acceso

Creó el núcleo original: **Usuario, Rol, Seccion, Permiso, RolPermiso y Sesion**. Añadió CI, nombres, contraseña, estado booleano, relaciones de rol/sección, claves primarias, índices únicos y claves foráneas. Fue la primera versión RBAC (control por roles y permisos).

2) 20260519033200_doble_verificacion

Creó **CodigoVerificacion** para el segundo factor. Cada código pertenece a un usuario y tiene fecha de creación, vencimiento y marca de uso.

3) 20260519034304_doble_verificacion

Añadió `correo` a Usuario y originalmente lo hizo único. Esto permitió usar correo como destino OTP.

4) 20260630160707_estructura_usuarios_notas_auditoria

Fue una ampliación mayor. Creó enumeraciones de grupos, estados, gestión, notas y tipos de sección; convirtió el estado booleano de Usuario a `EstadoUsuario`; amplió OTP y Sesion; agregó jerarquía organizacional; creó Curso, GestionAcademica, Nota, CertificadoNota, LogAcceso y AuditoriaAccion. También insertó secciones y cursos históricos iniciales.

> **Consecuencia histórica:** esta migración creó cursos llamados `1ER_ANIO`, `2DO_ANIO`, `3ER_ANIO`. El seed posterior creó "Primer Año", "Segundo Año" y "Tercer Año". Por eso existen cursos duplicados por orden. El frontend actual selecciona dinámicamente el curso que realmente contiene cadetes, sin hardcodear IDs.

5) 20260713201318_redisenio_cadete_responsable_academico

Separó los datos específicos del cadete fuera de Usuario. Creó **Cadete**, **JerarquiaUsuario**, **PermisoSeccionRol** e **HistorialPassword**. Añadió `primer_inicio`. Creó EstadoCadete y AreaAcademica. Esta separación evita mezclar datos generales de identidad con grado, arma, curso y egreso.

6) 20260713202824_redisenio_final_academico

Reemplazó ResponsableAcademico por **AsignacionAcademica**, incorporando el nivel OPERADOR/JEFE, vigencia y área. Añadió indicadores de módulos académico y documental en Usuario.

7) 20260713202950_redisenio_final_academico_historial

Permitió asignaciones de JEFE sin curso (`id_curso nullable`) y creó **HistorialCursoCadete** para conservar promociones y estados anteriores.

8) 20260713211022_agregar_refresh_historico_y_rtr

Creó **RefreshHistorico** para Refresh Token Rotation. Permite detectar reutilización de tokens, revocarlos y cerrar sesiones comprometidas. Añadió índice único para asignaciones de jefe sin curso.

9) 20260713220637_permitir_correo_compartido_cadetes

Eliminó la unicidad global de correo. Esto permite que varios cadetes de prueba compartan un buzón OTP. El CI continúa siendo la identidad única.

6. Esquema Prisma actual

Enumeraciones

Enum	Valores	Uso
GrupoUsuario	CADETES, ADMINISTRATIVOS, ADMINISTRATIVOS_ACADEMICOS	Clasifica usuarios.
EstadoUsuario	ACTIVO, INACTIVO, BLOQUEADO	Permite o impide acceso.
EstadoCadete	REGULAR, CONDICIONADO, DE_BAJA, EGRESADO	Situación académica/administrativa.
AreaAcademica	DISCIPLINA, ENTRENAMIENTO_FISICO, EVALUACIONES	Área de una asignación futura.
NivelResponsabilidad	OPERADOR, JEFE	Alcance de una asignación.
EstadoGestion	PLANIFICACION, ACTIVA, CERRADA	Ciclo anual.
EstadoNota	12 estados desde BORRADOR hasta PUBLICADO	Preparación del workflow futuro.
TipoSeccion	SISTEMA, ADMINISTRATIVA, ACADEMICA, DOCUMENTACION, MIXTA, FUTURA	Clasificación organizacional.

Modelos y finalidad

Modelo	Qué representa	Relaciones importantes
Usuario	Identidad común, CI, correo, hash bcrypt, estado y primer inicio.	Rol, Seccion, Cadete, sesiones, OTP, jerarquías, auditoría.
Cadete	Extensión 1:1 de Usuario con grado, arma, curso, ingreso, egreso y estado.	Usuario, Curso, HistorialCursoCadete.
HistorialCursoCadete	Registro permanente de curso/gestión/estado.	Cadete y Curso.
AsignacionAcademica	Responsabilidad administrativa preparada para áreas académicas.	Usuario y Curso opcional.
JerarquiaUsuario	Supervisor/subordinado con vigencia.	Dos relaciones hacia Usuario.
PermisoSeccionRol	Permiso contextual por rol y sección.	Rol, Seccion, Permiso.
HistorialPassword	Hashes anteriores para impedir reutilización.	Usuario.
Rol	Perfil funcional, por ejemplo ADMIN o CADETE.	Usuarios y RolPermiso.
Seccion	Unidad organizacional jerárquica.	Sección padre/hijas, usuarios y notas.
Curso	Año académico ordenado.	Cadetes, historial, notas y asignaciones.
Permiso	Acción autorizable: módulo + acción.	Roles y permisos contextuales.

RolPermiso	Tabla puente muchos-a-muchos.	Rol y Permiso; combinación única.
Sesion	Sesión iniciada, expiración, IP, agente y cierre.	Usuario y RefreshHistorico.
RefreshHistorico	Hash y estado de cada refresh token rotado.	Sesion.
CodigoVerificacion	Hash HMAC del OTP, expiración, uso e intentos.	Usuario.
GestionAcademica	Año y estado de gestión.	Notas y certificados futuros.
Nota	Estructura futura de calificación y workflow.	Cadete, responsable, gestión, sección y curso.
CertificadoNota	Resultado ponderado futuro.	Cadete y gestión.
LogAcceso	Eventos de login, OTP y seguridad.	Usuario opcional.
AuditoriaAccion	Trazabilidad de cambios con antes/después JSON.	Usuario ejecutor.

7. Backend NestJS: clases y responsabilidades

Arranque y núcleo

Clase/archivo	Función
main.ts	Crea Nest, activa ValidationPipe, filtro Prisma, CORS, Swagger y escucha `PORT` o 3001.
AppModule	Importa y conecta todos los módulos.
AppController / AppService	Endpoint raíz de comprobación.
PrismaModule / PrismaService	Cliente PostgreSQL compartido; conecta y desconecta con el ciclo Nest.

Infraestructura común

Clase	Qué hace
JwtAuthGuard	Lee Bearer token, verifica JWT, sesión activa, usuario y primer inicio.
PermisosGuard	Consulta permisos activos del rol en tiempo real y autoriza el endpoint.
PrismaExceptionFilter	Convierte errores Prisma en respuestas HTTP entendibles.
@Permisos	Declara permisos aceptados en un endpoint.
@UsuarioAutenticado	Extrae el usuario validado de la solicitud.

Módulo Autenticacion

AutenticacionController publica iniciar-sesion, verificar-codigo, refresh, logout, me y cambiar-contrasena. **AutenticacionService** valida bcrypt, aplica rate limit, genera OTP de seis dígitos, guarda su HMAC por diez minutos, pide el correo, crea sesiones y tokens, rota refresh tokens, detecta reutilización, registra logs y cambia contraseñas conservando historial.

DTO: LoginDto (CI/contraseña), VerificarOtpDto (CI/OTP), RefreshDto (refresh token) y CambiarContrasenaDto (actual/nueva).

Módulo Correo

CorreoService crea el transporter SMTP, verifica conexión y envía el HTML del OTP. Registra `messageId`, accepted y rejected. En EMAIL_TEST_MODE sólo muestra el OTP si NODE_ENV=development, pero intenta igualmente el envío real. Un fallo se propaga; no responde éxito falso.

Módulo Usuarios

UsuariosController: listar, detalle, crear, actualizar, estado, rol, sección, supervisor y restablecer contraseña. **UsuariosService**: bcrypt, CI único, creación transaccional con jerarquía, paginación, búsqueda, protección del único ADMIN, revocación de sesiones al cambiar privilegios y recuperación temporal.

DTO: CrearUsuarioDto; ActualizarUsuarioDto; FiltrarUsuariosDto; CambiarEstadoDto; CambiarRolDto; CambiarSeccionDto; CambiarSupervisorDto.

Módulo Cadetes

CadetesController/Service: listado paginado, búsqueda, creación atómica Usuario→Cadete→Historial, detalle por ID/CI, edición, cambio de curso, egreso e historial. Para primer año fuerza arma nula; para segundo/tercero exige arma. La transacción evita un Usuario huérfano si falla Cadete.

DTO: CrearCadeteDto, EgresarCadeteDto, ActualizarCadeteDto, CambiarCursoDto y FiltrarCadetesDto.

Roles, Permisos, Secciones y Cursos

Service	Reglas
RolesService	CRUD lógico, protege ADMIN, evita eliminar roles usados y reemplaza permisos en transacción.
PermisosService	CRUD lógico y nombres únicos.
SeccionesService	CRUD, árbol jerárquico, protección de secciones oficiales y dependencias.
CursosService	CRUD, orden, protección de cursos oficiales y conteo de cadetes para resolver duplicados históricos.

JerarquiasService

Administra supervisor/subordinado, evita auto-supervisión y ciclos, conserva fecha_fin al revocar y permite consultar supervisores, subordinados e historial.

AsignacionesAcademicasService

Lista, crea y revoca responsabilidades. OPERADOR requiere curso; JEFE no lleva curso y cubre el área completa. Evita duplicados y múltiples jefes activos del mismo ámbito. Es estructura administrativa; no procesa notas.

AuditoriaService

Registra acciones, datos antes/después, IP y user-agent; lista auditoría y logs de acceso con filtros.

Clases Module

Cada clase terminada en `Module` agrupa controller, service e imports. No contiene reglas; su función es decirle a Nest cómo inyectar dependencias.

8. Autenticación, OTP y seguridad

```
1. POST /autenticacion/iniciar-sesion {ci, contraseña}
2. findUnique(ci) → bcrypt.compare
3. verificar ACTIVO y correo
4. randomInt → OTP 6 dígitos
5. HMAC-SHA-256 → CodigoVerificacion (10 minutos)
6. await SMTP sendMail
7. POST /autenticacion/verificar-codigo
8. marcar OTP usado → Sesion + access token + refresh token
9. si primer_inicio=true → pantalla cambiar contraseña
10. nueva contraseña bcrypt + historial + cierre de sesiones
```

- CI inexistente ejecuta bcrypt ficticio para dificultar enumeración.
- Cinco fallos activan bloqueo temporal en memoria.
- OTP tiene máximo de intentos y se invalida al usarse.
- Si SMTP falla, el OTP queda invalidado y el endpoint falla.
- Refresh Token Rotation detecta token reutilizado.
- Los secretos viven en variables de entorno, no deben aparecer en documentación o commits.

9. Frontend Next.js

Páginas

Ruta	Función
/	Página pública institucional.
/iniciar-sesion	Credenciales y seis casillas OTP.
/cambiar-contrasena	Cambio obligatorio del primer inicio.
/panel	Resumen del usuario y permisos.
/acceso-restringido	Mensaje cuando falta autorización.
/modulos/control-acceso/usuarios	Administrativos, estados, supervisores y recuperación.
/modulos/control-acceso/cadetes	Todos/1.º/2.º/3.º/Egresados; CRUD, búsqueda y filtros.
/modulos/control-acceso/roles	Roles y matriz de permisos.
/modulos/control-acceso/permisos	Catálogo de permisos.
/modulos/control-acceso/secciones	Catálogo organizacional.
/modulos/control-acceso/asignaciones	Responsabilidades estructurales de usuarios.

Componentes

Componente	Función
LayoutProtegido	Compone barra lateral, superior y contenido autenticado.
BarraLateral	Menú condicionado por permisos, opción activa resaltada, hover y presión.
BarraSuperior	Identidad y salida.
FormularioUsuario	Alta de administrativos y administrativos académicos.
TablaUsuarios	Lista segura, estados, supervisor y recuperación.
NavbarPublico / FooterPublico	Navegación y pie públicos.
AccesoRestringido	Vista reutilizable de autorización denegada.

Servicios frontend

`api.ts` configura Axios y añade el token. Los servicios autenticacion, usuarios, cadetes, roles, permisos, secciones y asignaciones encapsulan URLs y tipos. Las respuestas paginadas usan `{data, meta}`; el estado de tabla recibe sólo `respuesta.data`.

Tipos

Las interfaces UsuarioSesion, UsuarioListado, Cadete, Curso, RespuestaCadetes, Rol, Permiso y Seccion reflejan contratos de API y evitan `any` en las operaciones principales.

10. Panel administrativo actual

Cadetes

- Conteos actuales: 116 primer año, 130 segundo, 117 tercero; total 363.
- Los IDs no están hardcodeados: se consultan cursos y se elige el curso relacionado con cadetes por orden.
- Primer año oculta arma y envía null.
- Segundo/tercero exigen Inf., Cab., Art., Ing., Com., Int., Mot., San., MB. u otra.
- Búsqueda por CI/nombre/apellido, filtro de estado, edición, suspensión/reactivación y recuperación.
- Historial y botón Egresar están ocultos actualmente por decisión de interfaz, aunque el backend conserva las capacidades.

Administrativos

Alta por grupo, rol y sección; listado paginado; estado; supervisor; recuperación de contraseña. Las acciones refrescan sólo los datos de la pantalla.

Roles y permisos

CRUD de catálogos y asignación/quita del arreglo completo de permisos de un rol. El guard consulta la relación real al autorizar.

Asignaciones

Permite configurar JEFE u OPERADOR por área y curso. Se mantiene para no rehacer estructura cuando llegue el módulo de notas.

11. Seed y estado de datos

El seed valida listas, crea roles, secciones, permisos, relaciones, cursos, gestión, administrador y 363 cadetes. Sus upserts de usuarios existentes no deben sobrescribir contraseñas. Para cadetes existentes actualiza datos identificativos, no su hash. La creación inicial usa bcrypt.

Conjunto	Cantidad/rango
Primer año	116, CI 100001–100116
Segundo año	130, CI 200001–200130
Tercer año	117, CI 300001–300117
Total	363

> El seed es idempotente, pero no debe usarse como mecanismo de recuperación de contraseñas. Las recuperaciones usan scripts/servicios específicos y entrada secreta.

12. Pruebas, ejecución y mantenimiento

Comandos backend

`npm run start:dev`	API con recarga.
`npm run build`	Compila Nest/TypeScript.
`npm test -- --runInBand`	Ejecuta pruebas Jest.
`npm run seed`	Ejecuta seed; usar con conocimiento del impacto.
`npm run auditar:auth`	Auditoría sanitizada.

Comandos frontend

`npm run dev`	Next en desarrollo.
`npm run build`	Compila y valida tipos/rutas.
`npm run start`	Sirve build de producción.

Pruebas actuales

Existen specs para app, autenticación, usuarios, roles, permisos y secciones. En la última verificación: 11 suites, 28 pruebas aprobadas; backend y frontend compilaron. Swagger permite pruebas manuales autenticadas.

Reglas de mantenimiento

- Nunca ejecutar migrate reset en una base con datos reales.
- No borrar migraciones aplicadas.
- Crear cambios estructurales con una migración nueva.
- No imprimir contraseñas, tokens u OTP fuera del modo autorizado.
- Usar transacciones cuando se escriben varias tablas.
- Refrescar una tabla después de mutar, no recargar todo el dashboard.

13. Estado actual y pendientes

Disponible

Login OTP JWT/RTR Primer inicio Usuarios 363 cadetes Roles Permisos Secciones Supervisores Asignaciones Auditoría

Preparado en datos, pero no desarrollado funcionalmente

Registro/visualización de notas, disciplina, entrenamiento físico, evaluaciones, matriz, aprobación académica, ponderación, certificados y publicación. Los modelos existen para evitar rediseñar usuarios y permisos, pero no equivalen a un módulo terminado.

Deuda técnica conocida

- Cursos históricos duplicados por nombre/orden. Hoy se resuelve dinámicamente; a futuro conviene una migración de consolidación cuidadosamente planificada.
- Ampliar cobertura automatizada para creación transaccional de cadetes y UI.
- Reemplazar algunos prompts administrativos por modales tipados y accesibles.
- Centralizar mensajes de error y paginación reutilizable.

14. Glosario

Término	Explicación sencilla
API	Puerta por la que frontend y backend intercambian información.
Endpoint	URL concreta de una operación.
DTO	Clase que define y valida lo permitido en una petición.
ORM	Traduce objetos TypeScript a operaciones SQL.
Migración	Cambio versionado de estructura de base de datos.
Seed	Carga controlada de catálogos y datos iniciales.
Hash	Transformación irreversible usada para proteger secretos.
JWT	Token firmado que identifica una sesión.
OTP	Código de un solo uso y corta duración.
Guard	Filtro de seguridad antes del controller.
Transacción	Grupo de cambios que se confirman todos o se revierten todos.
FK	Clave foránea que relaciona tablas y evita referencias inválidas.
CRUD	Crear, leer, actualizar y eliminar/desactivar.
Paginación	Entrega de resultados en páginas con data y meta.

Documento generado el 14 de julio de 2026 a partir del estado local del repositorio Plataforma EMSE. No contiene secretos ni credenciales.